Guion de presentación del Portal Vial IMSJ

Historia del proyecto y justificación de las decisiones técnicas

Duración sugerida 14 a 16 minutos

Estructura Apertura narrativa seguida de defensa técnica

Este guion presenta el proyecto como una secuencia de decisiones: partir de una necesidad concreta, reducir complejidad donde no aportaba valor y reforzar los puntos donde la consistencia, la seguridad y la operación sí lo exigían. La segunda parte transforma esa historia en argumentos técnicos verificables.

Indicaciones entre corchetes son sugerencias de ritmo y no se leen en voz alta.

Mapa de la presentación

Acto 1 La historia. Diapositivas 1 a 5. Presenta el problema, el criterio de diseño y el momento en que el proyecto encuentra una forma defendible.

Acto 2 Las decisiones. Diapositivas 6 a 12. Justifica tecnologías, arquitectura, datos, seguridad, despliegue, operación y verificación.

Cierre. Diapositiva 13. Resume la tesis y reconoce los límites que deben revisarse si cambia la escala.

Tesis de la defensa: la solución no intenta ser la más compleja ni la única posible. Busca ser proporcional al problema, comprensible para el equipo, consistente en sus datos y operable en un único servidor.

Acto 1 La historia del proyecto

Diapositiva 1 Una necesidad concreta

Tiempo 50 segundos

En pantalla *Nombre del proyecto, dos imágenes o íconos: ciudadanía y gestión interna.*

> [Mirar al público. Pausa breve.] Este proyecto no empezó con una discusión sobre frameworks o contenedores. Empezó con una necesidad más simple: ofrecer un portal de educación vial y, al mismo tiempo, darle al IMSJ una forma ordenada de gestionar contenidos, consultas, materiales y reservas.
>
> Desde el primer momento aparecieron dos públicos. Por un lado, la ciudadanía, que necesita acceder sin fricción. Por otro, el equipo de la institución, que necesita administrar información y conservar el control de lo que publica. El desafío fue unir esos recorridos sin convertir la solución en una infraestructura desproporcionada.

Idea que debe quedar La tecnología está al servicio de dos recorridos concretos: el público y el administrativo.

Transición *Para responder a esa necesidad, primero había que decidir qué complejidad era necesaria y cuál no.*

Diapositiva 2 Construir sin sobredimensionar

Tiempo 55 segundos

En pantalla *Esquema simple: portal público y panel administrativo conectados a una misma API.*

> La tentación podía ser separar todo desde el inicio: varios servicios, distintos servidores, herramientas de orquestación y capas adicionales. Pero el proyecto debía ser construido, explicado, probado y operado por un equipo pequeño.
>
> Por eso se adoptó un criterio que atraviesa toda la solución: agregar complejidad solo cuando resuelve un riesgo real. Se separaron las responsabilidades que sí necesitaban límites claros, como la interfaz, las reglas de negocio y los datos. En cambio, se evitó distribuir el sistema donde esa distribución solo habría agregado comunicación, configuración y fallos posibles.

Idea que debe quedar La sencillez no se planteó como ausencia de diseño, sino como proporcionalidad.

Transición *Ese criterio se volvió especialmente importante cuando el equipo revisó la forma de implementar el backend.*

Diapositiva 3 El punto de inflexión

Tiempo 1 minuto

En pantalla *Fecha 30 de septiembre y una flecha: Laravel hacia PHP nativo con capas.*

> El 30 de septiembre quedó registrada una decisión que cambió la forma de defender el proyecto: dejar de conservar Laravel y continuar con PHP nativo.
>
> No se hizo porque Laravel fuera una mala herramienta. Se hizo porque el equipo necesitaba mostrar el recorrido completo de una solicitud: cómo entra por una ruta, cómo el controlador valida, cómo el servicio aplica la regla de negocio y cómo el repositorio ejecuta SQL.
>
> La decisión redujo dependencias y volvió visible el funcionamiento interno, pero también trasladó responsabilidades al equipo. Las rutas, las validaciones, las migraciones y los controles de seguridad ya no podían darse por resueltos por un framework. Tuvieron que quedar explícitos y ser probados.

Idea que debe quedar Se ganó claridad pedagógica y control explícito, aceptando más responsabilidad de implementación.

Transición *A partir de ese cambio, la arquitectura dejó de ser una lista de tecnologías y pasó a ser una historia de responsabilidades.*

Diapositiva 4 De una decisión a un sistema

Tiempo 1 minuto y 5 segundos

En pantalla *Flujo visual: navegador, Apache y PHP, capas internas, MySQL y almacenamiento de archivos*.

> La solución tomó una forma concreta: dos interfaces separadas, una pública y otra administrativa; una API común; un backend organizado en controladores, servicios y repositorios; y una base MySQL para conservar relaciones, restricciones y transacciones.
>
> Luego se preparó esa aplicación para vivir fuera de la computadora de desarrollo. Docker describió el entorno, Compose coordinó aplicación y base de datos, y Ubuntu Server se definió como host de operación.
>
> La historia del proyecto, entonces, no es la de sumar herramientas. Es la de establecer límites: qué ve el navegador, dónde se aplica una regla, quién controla un permiso, qué debe persistir y cómo puede reconstruirse el sistema.

Idea que debe quedar Cada componente existe para ubicar una responsabilidad y facilitar su revisión.

Transición *Con esa historia completa, podemos formular la tesis técnica que guía la defensa*.

Diapositiva 5 La tesis de la solución

Tiempo 45 segundos

En pantalla *Una frase central: proporcional, explicable, consistente y operable*.

> [Bajar el ritmo.] La solución no se presenta como la única arquitectura válida. Se presenta como una arquitectura adecuada para este alcance.
>
> Es proporcional porque trabaja con un host y dos servicios. Es explicable porque el recorrido del código es visible. Es consistente porque las reglas críticas se sostienen con transacciones y restricciones. Y es operable porque contempla configuración, salud, registros, copias y restauración.
>
> A partir de aquí voy a justificar técnicamente cómo se sostiene cada una de esas afirmaciones.

Idea que debe quedar La defensa no depende de afirmar superioridad absoluta, sino adecuación al contexto.

Transición *Primero, la base tecnológica y las versiones elegidas*.

Acto 2 Las justificaciones técnicas

Diapositiva 6 Tecnologías y versiones

Tiempo 1 minuto y 20 segundos

En pantalla *Ubuntu Server 26.04 LTS, PHP 8.5, MySQL 8.4, Apache, Docker Engine y Compose v2.*

Ubuntu Server 26.04 LTS se eligió como host porque la aplicación no necesita un entorno gráfico. La administración puede realizarse por SSH, terminal y Cockpit, y una rama LTS reduce la frecuencia de cambios de distribución. La decisión no invalida Ubuntu 24.04 si la institución ya tuviera esa versión estandarizada.

PHP 8.5 se mantiene igual entre desarrollo, Composer y la imagen del contenedor. El argumento principal es la coherencia de una rama soportada, no una supuesta mejora automática de rendimiento. MySQL 8.4 se conserva como rama LTS y con InnoDB porque el dominio necesita relaciones, transacciones, claves foráneas y bloqueo de filas.

Apache y PHP se ejecutan en un mismo servicio web para resolver archivos, reescritura y ejecución con una configuración acotada. Docker y Compose describen los dos servicios, la red, los volúmenes y la salud sin introducir un orquestador de clúster.

Una precisión importante: etiquetas como php:8.5-apache o mysql:8.4 fijan una rama, pero no siempre el mismo binario. Para reproducibilidad deben registrarse la revisión y el digest efectivamente desplegados.

Idea que debe quedar Las versiones se justifican por soporte, coherencia y mantenimiento; no por ser simplemente las más nuevas.

Diapositiva 7 Arquitectura y organización del código

Tiempo 1 minuto y 20 segundos

En pantalla *Diagrama de capas: entrada, controladores, servicios, repositorios, modelos y núcleo.*

El backend es un monolito modular. El sistema comparte usuarios, datos y transacciones, por lo que dividirlo en microservicios habría agregado autenticación entre servicios, fallos de red y coordinación distribuida sin una ventaja demostrada para ocho módulos y un equipo pequeño.

Dentro del monolito sí hay separación. public/index.php centraliza la entrada y las rutas. Los controladores interpretan la solicitud y verifican identidad o rol. Los servicios aplican reglas y coordinan transacciones. Los repositorios concentran SQL. Los modelos y el núcleo reúnen comportamiento común.

Las dos interfaces usan HTML, CSS y JavaScript sin un framework de frontend. Para formularios, listados y contenido, esto mantiene visible la relación entre eventos y peticiones y evita una compilación adicional. La separación de carpetas facilita el mantenimiento, pero no se considera un control de acceso: la autorización siempre se vuelve a comprobar en el servidor.

Idea que debe quedar Monolito no significa código mezclado; significa una unidad de despliegue con responsabilidades internas separadas.

Diapositiva 8 Datos e integridad

Tiempo 1 minuto y 25 segundos

En pantalla *Ejemplo de dos personas intentando reservar el último cupo de una misma franja*.

La elección de MySQL se vuelve crítica en el caso del último cupo. Si dos personas consultan al mismo tiempo, ambas pueden ver disponibilidad. Por eso no alcanza con validar en el navegador ni con hacer una consulta antes del alta.

El servicio inicia una transacción, bloquea la franja, comprueba duplicación y capacidad, crea la reserva y recién entonces confirma. Las restricciones del esquema respaldan esa lógica. En operaciones administrativas, la modificación y su auditoría también se coordinan dentro de la misma transacción.

PDO y las consultas preparadas mantienen visible el SQL y separan la estructura de los valores. No se usa un ORM porque ocultaría parte del recorrido que el equipo necesita explicar, aunque el costo sea escribir y mantener más consultas.

El esquema inicial y las migraciones incrementales se mantienen separados: database.sql crea una instalación nueva y migrar.php actualiza una existente sin borrar volúmenes. Si la evolución crece, un historial numerado de migraciones sería el siguiente paso razonable.

Idea que debe quedar La integridad se protege en la transacción y en la base, no solo en la interfaz.

Diapositiva 9 Sesiones, permisos y archivos

Tiempo 1 minuto y 20 segundos

En pantalla *Tres bloques: token revocable, autorización en servidor y archivos fuera de la raíz pública*.

La sesión usa un token opaco aleatorio. El servidor guarda su hash, la cuenta asociada y el vencimiento. Esto permite revocar la sesión al cerrar, reemplazarla al iniciar nuevamente y comprobar que la cuenta siga activa. El costo es consultar la base en cada validación; para un único backend que ya depende de MySQL, ese costo simplifica el control.

Las contraseñas usan bcrypt con costo 12. El token se conserva en sessionStorage, lo que limita su persistencia a la pestaña, aunque no elimina el riesgo de XSS porque el JavaScript del origen puede leerlo.

Los adjuntos se validan por tamaño, tipo detectado y contenido básico, reciben nombres aleatorios y se guardan fuera de public. La descarga pasa por una ruta controlada. Esto reduce exposición, pero no equivale a un antivirus completo.

Además, el DocumentRoot se limita a public y las respuestas sensibles no se reutilizan en caché. Los recursos estáticos sí pueden conservarse por un período corto para reducir transferencias.

Idea que debe quedar La seguridad se construye con controles complementarios y límites explícitos; ninguna medida aislada resuelve todo.

Diapositiva 10 Despliegue y dimensionamiento

Tiempo 1 minuto y 20 segundos

En pantalla *Dos contenedores sobre una VM: app y db. Recursos iniciales: 2 vCPU, 4 GB RAM y 30 GB SSD o NVMe.*

Compose separa aplicación y base de datos. Así puede reconstruirse el contenedor de aplicación sin sustituir el volumen de MySQL. Los datos, adjuntos y caché también tienen ciclos de vida diferenciados. Un volumen aporta persistencia, pero no es una copia de seguridad.

El dimensionamiento inicial de 2 vCPU, 4 GB de RAM y 30 GB de disco parte de solicitudes breves, recursos estáticos, dos contenedores y hasta 90 usuarios con períodos de lectura. No se interpreta como 90 procesos PHP activos de forma permanente.

La VM debe ser ampliable y la hipótesis debe validarse con carga, latencia, errores y consumo real. Un 70 por ciento sostenido es un umbral para revisar capacidad, no una garantía de rendimiento.

Kubernetes no se incorporó porque el proyecto tiene un host y dos servicios. Sus ventajas de clúster no compensan aquí el costo administrativo. Si aparecieran múltiples nodos o exigencias fuertes de disponibilidad, esa decisión debería revisarse.

Idea que debe quedar La infraestructura se dimensiona como punto de partida medible, no como promesa definitiva.

Diapositiva 11 Operación, red y recuperación

Tiempo 1 minuto y 20 segundos

En pantalla *Secuencia operativa: salud, registros, backup diario, copia externa y restauración aislada.*

La operación incluye controles de salud, política de reinicio, rotación de registros, usuarios de servicio, raíz de solo lectura, tmpfs y capacidades reducidas. Estas medidas limitan fallos y privilegios, pero no sustituyen la actualización de imágenes.

La configuración se mantiene fuera del código mediante variables. La primera cuenta administrativa se crea por línea de comandos, evitando credenciales de demostración incorporadas al esquema.

El backup realiza un volcado lógico de MySQL, coordina SQL y adjuntos deteniendo la aplicación, comprime el conjunto y registra un manifiesto. La pausa es el compromiso de una estrategia simple y consistente. Cron reutiliza herramientas del servidor y la restauración se ensaya en otro proyecto Compose para reducir el riesgo de sobrescribir el entorno principal.

En red hay una salvedad que conviene expresar con claridad: los puertos publicados por Docker pueden eludir reglas habituales de UFW. La restricción efectiva debe verificarse con filtrado compatible con Docker y con pruebas desde redes autorizadas y no autorizadas.

Idea que debe quedar Operar significa poder observar, respaldar y reconstruir; no solo lograr que el sistema arranque.

Diapositiva 12 Pruebas, evidencia y límites

Tiempo 1 minuto y 10 segundos

En pantalla *Pirámide o lista: integración, auditoría API, análisis estático y exploración dinámica*.

La verificación combina técnicas porque cada una responde una pregunta diferente. La integración HTTP y MySQL comprueba contrato y persistencia. La auditoría API revisa accesos y entradas seleccionadas. El análisis estático y la exploración dinámica producen hallazgos que deben interpretarse.

El informe del 21 de octubre registró 192 comprobaciones sobre 46 endpoints. Ese resultado acredita la versión ensayada, no cualquier cambio posterior. Las modificaciones de categorías, validación e interfaz requieren su propia revalidación.

La defensa técnica también debe reconocer cuándo revisar la solución: más volumen puede exigir paginación y filtros de servidor; varios hosts, contadores compartidos; mayor disponibilidad, otra infraestructura; y más cambios de esquema, una herramienta formal de migraciones.

Idea que debe quedar La evidencia tiene alcance y fecha; una prueba histórica no reemplaza la verificación de la versión actual.

Transición *Con estas decisiones justificadas, el cierre vuelve a la idea que organizó todo el proyecto*.

Cierre

Diapositiva 13 Una solución que puede explicarse y sostenerse

Tiempo 55 segundos

En pantalla *Volver a la frase: proporcional, explicable, consistente y operable.*

> [Pausa breve.] El valor del Portal Vial IMSJ no está en haber acumulado tecnologías. Está en que cada decisión responde a una necesidad concreta.
>
> El PHP nativo hace visible el recorrido. Las capas ordenan las responsabilidades. MySQL y las transacciones protegen la consistencia. Los tokens revocables, los permisos y el manejo de archivos reducen exposición. Docker y Compose vuelven reconstruible el entorno. Los respaldos y las pruebas permiten sostenerlo y revisar sus límites.
>
> La conclusión es simple: se eligió una solución que el equipo puede comprender, defender y operar hoy, sin presentar sus decisiones actuales como reglas permanentes. Si cambia el problema, también deberán cambiar las decisiones. [Mirar al público.] Muchas gracias.

Idea que debe quedar La arquitectura es adecuada porque sus decisiones están alineadas con el alcance y sus límites están reconocidos.

Respuestas breves para la defensa

Por qué PHP nativo y no Laravel

Porque el objetivo fue hacer explícito el recorrido de una solicitud y facilitar la documentación y defensa. Se aceptó el costo de implementar rutas, validaciones y seguridad de forma visible. No se afirma que Laravel sea inadecuado.

Por qué un monolito y no microservicios

Porque el sistema comparte datos y transacciones, se despliega como una unidad y tiene ocho módulos. Microservicios agregarían autenticación interna, red y coordinación distribuida sin una necesidad demostrada.

Por qué MySQL y no archivos JSON

Porque el dominio necesita claves foráneas, restricciones, transacciones y bloqueo de filas para resolver concurrencia, especialmente en reservas.

Por qué token opaco y no JWT

Porque el token opaco puede revocarse de forma directa y permite comprobar el estado actual de la cuenta. El costo de consultar MySQL es aceptable en un único backend.

Por qué Docker Compose y no Kubernetes

Porque hay un host y dos servicios. Compose cubre red, volúmenes, configuración y salud con menor carga operativa. Kubernetes se reconsideraría con múltiples nodos o requisitos de alta disponibilidad.

La versión queda totalmente congelada

No. Las etiquetas fijan una rama, no siempre una revisión exacta. Para una entrega reproducible se deben registrar revisión y digest, actualizar parches y revalidar.

UFW alcanza para proteger el puerto publicado

No necesariamente. Docker puede alterar el recorrido del filtrado. La restricción debe verificarse con reglas compatibles con Docker y pruebas reales desde redes permitidas y no permitidas.

Un volumen Docker es un backup

No. El volumen conserva datos fuera del contenedor, pero puede borrarse o corromperse. El respaldo debe incluir base, adjuntos, configuración y código compatible, y debe existir una copia externa.

Las 192 pruebas garantizan la versión actual

Garantizan únicamente el alcance y la versión ensayada el 21 de octubre. Los cambios posteriores deben ejecutarse y documentarse nuevamente.